Chapter 2 練習

作答時寫出位元欄位、register 編號、effective address 與必要前提。沒有指定 ISA release 時，以本章介紹的 MIPS32 core instruction 為準，忽略 pseudo-instruction。採 pre-Release-6 MIPS32 的一個控制轉移 delay slot，範例以 `nop` 明示；`jal` 的 `$ra=PC+8`。

CH02-Q01 編碼

1. 將 `sll $t0, $t1, 3` 編成 32-bit word，以十六進位表示。
2. 將 `addiu $s0, $sp, -16` 編成 32-bit word，以十六進位表示。
3. 指出兩個 word 的 format、opcode、主要欄位與 immediate extension 方式。

CH02-Q02 二補數與溢位

令 MIPS register 為 32 bits。

1. 將 `0xfffffff6` 分別解釋為 unsigned 與 signed decimal。
2. 執行 signed `0x7fffffff + 1`，指出 result bits 與是否 overflow。
3. 執行 unsigned `0xffffffff + 1`，指出 result bits 與是否 overflow。
4. 若同一個 `0xffffffff` 被程式當成 unsigned counter 與 signed balance，分別和 1 比較時要用哪條 MIPS comparison？哪些結果來自型別語意，而不是 bit pattern？

CH02-Q03 迴圈與不變量

下列片段把陣列中的 signed integers 取絕對值後累加到 `$v0`。執行前明定 `$a0` 指向 word-aligned array，`$a1=n` 是陣列元素數，`0 <= n <= array length`，且 `$a0 + 4(n-1)` 不越過 32-bit 位址空間。初始化移除未定義值。

```
addiu $t0, $zero, 0
addu  $v0, $zero, $zero
loop:
beq  $t0, $a1, done
nop
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
slt  $t3, $t2, $zero
beq  $t3, $zero, skip
nop
sub  $t2, $zero, $t2
skip:
addu $v0, $v0, $t2
addiu $t0, $t0, 1
j    loop
nop
done:
```

Chapter 2 練習 （續）

1. 寫出每次 loop 進入 body 時成立的不變量。
2. 使用初始化、branch condition、delay slot 與每輪 `addiu`，證明條件允許時 loop 恰好執行 `$a1` 次。
3. 找出 `$t2 = 0x80000000` 時的問題。
4. 若累加可能超出 signed 範圍，哪些 instruction 的 overflow 或 modulo 行為會影響結果？

CH02-Q04 巢狀呼叫與 stack frame

函式 `outer` 接收一個 argument，需要保存 `$s0`，並在計算後呼叫 `inner`。`outer` 的 frame 大小為 32 bytes。只要交出必要概念 sequence，不必完整展開 `inner`。

1. 依 caller-saved、callee-saved 規則，列出 `outer` 至少應保存哪些狀態。
2. 寫出 entry、before-call、after-call、exit 的 stack frame 操作。
3. 若 `outer` 新增第二次 `jal inner`，frame 需要如何處理？
4. 若 `inner` 故意破壞 `$s0` 且不 restore，違反哪個 convention？

CH02-Q05 目標位址

假設 32-bit byte-addressable MIPS，instruction 位址為 4-byte aligned。

1. 某 `beq` 的 next PC 是 `0x00400010`，imm16 為 `0xfffa`。求 branch target。
2. 某 J-type 的 next PC 是 `0x0abcdef4`，target26 欄位為 `0x0004000`。求 target。
3. 第二題的 jump 能否直接跳到 `0x1abcdef4`？說明 region 限制。
4. Branch offset 若以 `0x0002` 表示，代表多少 bytes？

CH02-Q06 原子操作

共享 counter 的位址在 `$s0`。兩個 cores 各執行一次以下 sequence。

```
retry:
ll   $t0, 0($s0)
addiu $t0, $t0, 1
sc   $t0, 0($s0)
beq  $t0, $zero, retry
nop
```

Chapter 2 練習 （續）

1. 為什麼只寫 `lw`、`addiu`、`sw` 不足以保證 counter 增加兩次？
2. 列出 `sc` 失敗後程式能觀察到的狀態。
3. 若 `ll` 與 `sc` 之間插入一次 `sw` 到同一 address，會如何？
4. 此機制是否保證無饑餓？是否需要 atomic instruction 之外的 fairness 設計？

CH02-Q07 工具鏈

專案有 `main.o`、`math.o` 與 shared library `libvec.so`。`main.o` 呼叫 `dot`，`math.o` 再呼叫 shared library 中的 `axpy`。

1. 指出 assembler、linker、dynamic loader 各自處理哪些 unresolved reference。
2. 為什麼 object file 需要 relocation record？
3. `main.o` 載入時的 virtual address 與 file offset 為何不能當成同一種位址？
4. 如果 `main.o` 的 instruction 使用 PC-relative call，shared library 被放到較遠位址時要考慮什麼？

CH02-Q08 ISA 與實作

工作負載在版本 A 執行 1.8×10^9 instructions，平均 CPI 1.20，frequency 3.0 GHz。版本 B 新增一條較複雜 instruction，使 instruction count 降為 1.5×10^9，但平均 CPI 升為 1.55，frequency 仍為 3.0 GHz。

1. 分別計算 CPU time。
2. 以 A 為 baseline 求 B 的 speedup。
3. 為何不能只因為 B 的 instruction count 少 16.7% 就宣告 B 較快？
4. 如果 B 的 frequency 提升到何值，才能使其 CPU time 與 A 相同？

CH02-Q09 多格式解碼與控制轉移

使用本章的 classic MIPS32 模型，每個控制轉移後有一個 delay slot。

1. `0x8fa8000c` 是何種格式與 instruction？求 destination、base 與 effective address。
2. `0x1109fffc` 位於 `0x00400020`。解碼兩側 operands 與 immediate，並分別列出 taken、not taken 的完整 PC sequence。
3. `0x0c100140` 位於 `0x00400070`。求 target、delay-slot PC 與 `$ra`。
4. 若把第 2 題的 `imm16` 當成 byte displacement，而不是 instruction displacement，會得到哪個錯誤 target？

Chapter 2 練習 （續）

CH02-Q10 資料布局、排序與成本

考慮

```
struct row { int64_t id; int32_t vals[4]; int64_t sum; };
```

`rows` 的 base 是 `0x3000`，已在 `%rdx`。另有 `int i=5, j=3;` 在 x86-64 中分別使用 `%rdi` 與 `%esi`。

1. 求 `sizeof(struct row)`、`vals` 的 offset 與 `rows[5].vals[3]` 的 byte address。
2. 寫出可計算該 address 的 `movslq`、`salq`、`leaq` 序列，並說明為何 `lea` 不會觸發 memory read。
3. 對 `v=[3,1,2]` 執行本章的 exchange sort，列出每次 inner-loop comparison 與 array state。
4. 在穩態、相同 word size 與固定 delay slot 的模型中，array-index clear 每次 iteration 有 7 個指令位置，pointer clear 有 5 個。說明這兩項計數排除了哪些成本，並指出 compiler strength reduction 為何可能讓兩份 C 原始碼得到相近的 machine code。

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study